

Supplementary Material for “Beyond Predefined Sinks: Security-Aware Dependency Analysis for LLM Agents”

ANONYMOUS AUTHOR(S)

1 AgentSecBench Repository Inventory and Collection Criteria

AGENTSECBENCH contains 67 publicly available LLM-agent repositories spanning 11 agent ecosystems. Corpus construction was performed independently of AGENTSECBENCH output. We retained repositories with substantive agent logic and analyzable source code, including MCP-based systems, coding agents, framework-based applications, autonomous agents, and multi-agent platforms. We excluded duplicates, mirrors, trivial examples, and repositories unsuitable for reproducible source-level analysis.

Each retained repository is identified by its repository coordinate and an immutable Git commit. Candidate extraction, graph construction, reproduction, and evaluation use that same frozen snapshot. The machine-readable frozen corpus manifest in the anonymous replication package provides the complete row-level repository inventory and commit identifiers; the PDF reports the aggregate composition in Table 1 rather than duplicating the long manifest.

The nine repositories in the remaining-language group primarily use C#, Rust, Go, PHP, or Shell. Repository inclusion is not conditioned on candidate counts, graph construction outcomes, reproduction outcomes, vulnerability dispositions, or model-generated judgments. The frozen manifest is therefore an input to the evaluation rather than an output selected by the analyzer.

2 Framework Normalization and Matching Rules

Agent frameworks expose tools and capabilities through heterogeneous source constructs. The adapter layer maps a matched construct to the normalized record

$$\mathcal{N} = \langle F, R_s, Y, P_s, Q \rangle, \quad (1)$$

where F is the framework family, R_s is the semantic role, Y is the associated callable or program symbol, P_s is the set of exposed parameters, and Q records the matching evidence and confidence. Normalization determines how source constructs are represented; it does not assign vulnerability labels.

Adapters apply frozen framework-specific selectors to decorators, calls, registrations, wrappers, workflow nodes, or tool classes. When available, module qualification and imported aliases are normalized before matching. A successful match records the construct form, associated symbol, source location, exposed parameters, and framework provenance. Ambiguous or unsupported constructs remain available to the language-level analysis but are not promoted to framework-specific evidence solely from a security label, model judgment, or reproduction outcome.

Framework normalization is independent of candidate-generation and graph schema decisions. Adding an adapter extends semantic recognition without changing the candidate-centered SECURITY-ADG representation. The frozen adapter registry and its tests are part of the complete method implementation to be included in the anonymous replication package.

3 Generative-AI-Assisted Candidate Triage

We evaluated a frozen panel of five model raters over 150 candidate tasks. Each rater processed every task independently in a fresh single-turn request. Raters did not receive scanner confidence, detector identity, sampling weight, human annotations, or another model’s response. Tools, browsing,

Table 1. Aggregate composition of the frozen AGENTSECBENCH corpus. Candidate counts characterize the current analyzer and are not vulnerability counts.

Corpus property	Value
Frozen repositories	67
Agent ecosystems	11
Source files	37,542
Source size	291.2 MB
Primarily Python repositories	51
Primarily TypeScript repositories	7
Repositories in other primary languages	9
Repositories with current-scan candidates	65
Current full-corpus candidates	23,866

Table 2. Framework construct families represented by the frozen adapter registry. Concrete selectors are versioned with the implementation.

Framework family	Construct family	Normalized interpretation
MCP / FastMCP	Tool decorators and server-side tool registration	Agent-facing callable and exposed parameters
LangChain	Tool decorators, callable wrappers, and toolkit exports	Tool capability associated with its callable
CrewAI	Tool declarations and tool collections attached to agents or tasks	Agent-available capability
AutoGen	Function or tool registration between conversational agents and executors	Registered callable capability
OpenAI Agents SDK	Function-tool wrappers and tools attached to an agent	Agent-facing callable capability
Semantic Kernel	Kernel-function declarations and plugin methods	Kernel-exposed callable capability
LlamaIndex	Function/query tool wrappers and tool collections	Agent- or query-facing callable capability

retrieval, memory, and code execution were disabled. Responses were required to follow a fixed JSON schema and were validated locally. Transport and schema failures were recorded rather than silently discarded.

Four raters were accessed through OpenRouter using the exact routing slugs openai/gpt-5-mini, anthropic/claude-sonnet-4.6, google/gemini-3.5-flash-lite, and qwen/qwen3-max. Provider fallbacks were disabled, parameter support was required, and the run recorded the returned model identifier, generation identifier, upstream provider, token usage, cost, and system fingerprint. The fifth rater, deepseek-v4-pro, was accessed through the official DeepSeek API rather than OpenRouter. The completed panel contains 750 valid task–rater responses (150 tasks times five raters).

Table 3. Frozen five-model pre-annotation panel.

Rater	Exact identifier and interface	Inference and output settings
M1	openai/gpt-5-mini; OpenRouter	Medium reasoning; reasoning excluded; JSON Schema
M2	anthropic/claude-sonnet-4.6; OpenRouter	Medium reasoning; reasoning excluded; JSON Schema
M3	google/gemini-3.5-flash-lite; OpenRouter	Medium reasoning; reasoning excluded; JSON Schema
M4	qwen/qwen3-max; OpenRouter	Provider-default reasoning; JSON Schema
M5	deepseek-v4-pro; DeepSeek official API	Thinking enabled, high; reasoning not stored; JSON object with local schema validation

The panel was used only for pre-annotation and disagreement-based triage. Inter-model agreement was not interpreted as human inter-annotator agreement or as evidence of semantic correctness. Panel responses were excluded from SECURITY-ADG construction and did not establish behavior or vulnerability ground truth.

For M1–M4, requests used <https://openrouter.ai/api/v1/chat/completions> with `allow_fallbacks=false`, `require_parameters=true`, and `data_collection=deny`. Zero-data-retention endpoints were preferred, but the recorded run explicitly allowed non-ZDR endpoints under the data-collection-denial policy. The recorded upstream providers were OpenAI, Anthropic, Google AI Studio, and Alibaba, respectively, and provider consistency checks passed. For M5, requests used <https://api.deepseek.com/chat/completions> with a separate API credential. Because DeepSeek’s JSON mode did not enforce the supplied JSON Schema, its outputs underwent the same local schema validation as the other responses. No API keys, account identifiers, billing records, or author-identifying generation URLs are included in the artifact.

4 Held-Out Context-Preservation Study

We evaluate three representation projections over nine held-out evidence-backed cases: five validated by controlled local monkeypatch proofs and four by source-level construction proofs that did not execute the external effect. Each case-level reference contains independently supported context dimensions for operation, external effect, dependency path, trust boundary, and guard posture. The four cases recorded as `no_guard_confirmed` contain an explicit negative guard-posture observation rather than an absent or unknown field; the guard-posture dimension therefore remains in the denominator. Positive guard-visibility credit is awarded only when concrete guard evidence is exposed by the representation. These measurements concern representation retention, not vulnerability-detection accuracy.

The micro totals reproduce the values reported in the main paper: $9/45 = 20.0\%$, $18/45 = 40.0\%$, and $41/45 = 91.1\%$. SECURITY-ADG exposes concrete guard evidence in all five guarded cases. The four remaining reference records explicitly report `no_guard_confirmed`; because the graph does not emit a positive guard node for these cases, they contribute no guard-retention credit.

5 Corpus-Wide Characterization and Scalability

This section provides the detailed corpus-wide characterization underlying RQ1 in the main paper. All measurements in this section use the *current full-corpus run*: the current candidate extractor applied to all 67 frozen repositories in AGENTSECBENCH. This population is distinct from the

Table 4. Evidence provenance for the held-out context-preservation cases. Repository identifiers are retained to make the frozen evidence auditable; no row is a vulnerability claim.

Case	Repository	Behavior	Evidence mode	Guard posture
HSQ-0014	dddabtc/winremote-mcp	shell command execution	local monkeypatch proof	no_guard_confirmed
HSQ-0087	dddabtc/winremote-mcp	application launch	local monkeypatch proof	no_guard_confirmed
HSQ-0089	dddabtc/winremote-mcp	desktop notification script	local monkeypatch proof	guarded
HSQ-0091	dddabtc/winremote-mcp	network diagnostic process	local monkeypatch proof	no_guard_confirmed
HSQ-0103	dddabtc/winremote-mcp	service control	local monkeypatch proof	guarded
HSQ-0211	nexu-io/html-anything	agent CLI discovery process	source-level construction proof	no_guard_confirmed
HSQ-0298	nexu-io/html-anything	external content file write	source-level construction proof	guarded
HSQ-0300	nexu-io/html-anything	external tarball extraction	source-level construction proof	guarded
HSQ-0302	nexu-io/html-anything	validated skill file write	source-level construction proof	guarded

Table 5. Case-level representation retention. Each case has five applicable reference dimensions: operation, effect, dependency, trust boundary, and guard posture.

Case	Guard posture	Applicable	Sink-only	Simplified ADG	Security-ADG
HSQ-0014	no_guard_confirmed	5	1/5	2/5	4/5
HSQ-0087	no_guard_confirmed	5	1/5	2/5	4/5
HSQ-0089	guarded	5	1/5	2/5	5/5
HSQ-0091	no_guard_confirmed	5	1/5	2/5	4/5
HSQ-0103	guarded	5	1/5	2/5	5/5
HSQ-0211	no_guard_confirmed	5	1/5	2/5	4/5
HSQ-0298	guarded	5	1/5	2/5	5/5
HSQ-0300	guarded	5	1/5	2/5	5/5
HSQ-0302	guarded	5	1/5	2/5	5/5
Micro total	–	45	9/45	18/45	41/45

Table 6. Micro-averaged context completeness and positive guard visibility.

Representation	Retained	Completeness	Guard visibility
Sink-only	9/45	20.0%	0/5
Simplified ADG	18/45	40.0%	0/5
Security-ADG	41/45	91.1%	5/5

historical 531-candidate frozen pilot cohort and the 508 candidates obtained by re-running the current scanner on the same five pilot repositories.

Table 7. Aggregate evidence recovered in the current full-corpus run. Rates use all 23,866 static candidates as the denominator. Candidate counts are not vulnerability findings.

Evidence or artifact	Count	Rate
Framework-specific evidence	749	3.14%
Source-to-operation dependency evidence	9,821	41.15%
Potential guard evidence	3,075	12.88%
Generated SECURITY-ADG artifacts	23,866	100.00%

The current full-corpus run identifies 23,866 static security-sensitive operation candidates from 7,214 source files. Sixty-five of the 67 repositories contain at least one candidate. All 67 repositories complete candidate extraction and graph construction without reported skips or failures. Candidate counts in this section are analysis populations and must not be interpreted as vulnerability counts.

5.1 Aggregate Security-ADG Evidence

Table 7 summarizes the evidence exposed by the current full-corpus analysis. For every candidate, AGENTSECGRAPH emits one candidate-centered SECURITY-ADG artifact linked to its candidate identifier and source provenance.

The exported corpus-wide aggregates contain identical candidate counts for source provenance, dependency paths, and trust-crossing fields: 9,821 candidates in each case. These exported counts do not establish that the three fields were recovered by independent analytical events. We therefore report the consolidated *source-to-operation dependency evidence* measure in Table 7 rather than treating the three identical counts as independent results.

The current extraction rules recover no explicit privilege-context evidence in this run. This does not imply that the analyzed programs execute without privilege; rather, AGENTSECGRAPH records privilege context only when it can be derived from supported source-level constructs. Likewise, every candidate receives an abstract external-effect node based on its operation category. The resulting 100% external-effect-node coverage is therefore a schema property rather than evidence that the modeled effect occurred at runtime.

5.2 Evidence by Operation Category

Table 8 reports candidate counts and the frequency of recovered dependency, guard, and trust-crossing fields for each operation category. The category-level percentages characterize the current analyzer and corpus only. They do not estimate vulnerability prevalence or category-specific security risk.

Dependency evidence is strongly category-dependent. It is recovered for 84.14% of filesystem-write candidates and 65.28% of filesystem-read candidates, compared with 11.28% of external-tool invocations and 1.87% of browser-control candidates. Guard evidence is likewise uneven: 51.29% of filesystem-delete candidates and 41.82% of dynamic-code-execution candidates contain potential guard evidence, whereas only 1.37% of external-tool invocations do.

These differences should not be interpreted as evidence that one operation category is more vulnerable than another. They reflect the interaction among the current source-analysis rules, the coding patterns in the frozen corpus, and the evidence that can be recovered statically.

As noted above, dependency and trust-crossing counts are identical in the exported corpus-wide aggregate. The trust-crossing column is retained in Table 8 for auditability of the archived results, but we do not interpret it as an independently validated trust-boundary recovery rate.

Table 8. Corpus-wide evidence by security-sensitive operation category. Percentages are relative to the number of candidates in each row.

Category	Candidates	Dependency	Guard	Trust-crossing
browser_control	588	11 (1.87%)	13 (2.21%)	11 (1.87%)
command_execution	2,722	933 (34.28%)	315 (11.57%)	933 (34.28%)
credential_access	2,548	251 (9.85%)	454 (17.82%)	251 (9.85%)
database_access	1,304	106 (8.13%)	36 (2.76%)	106 (8.13%)
dynamic_code_execution	55	24 (43.64%)	23 (41.82%)	24 (43.64%)
external_tool_invocation	5,344	603 (11.28%)	73 (1.37%)	603 (11.28%)
filesystem_delete	583	235 (40.31%)	299 (51.29%)	235 (40.31%)
filesystem_read	4,182	2,730 (65.28%)	1,062 (25.39%)	2,730 (65.28%)
filesystem_write	5,013	4,218 (84.14%)	632 (12.61%)	4,218 (84.14%)
message_or_email_send	164	70 (42.68%)	32 (19.51%)	70 (42.68%)
network_access	1,157	553 (47.80%)	121 (10.46%)	553 (47.80%)
permission_or_auth_change	206	87 (42.23%)	15 (7.28%)	87 (42.23%)
Total	23,866	9,821 (41.15%)	3,075 (12.88%)	9,821 (41.15%)

Table 9. Scalability measurements from one completed full-corpus run. The measurements characterize the observed execution environment and are not presented as cross-system performance estimates.

Metric	Observed result
Frozen repositories processed	67 / 67
Repositories with candidates	65
Candidate-bearing source files	7,214
Static candidates	23,866
Generated SECURITY-ADG artifacts	23,866
Candidate extraction time	1,508.456 s
Candidate extraction throughput	15.821 candidates/s
Graph construction time	1,538.164 s
Graph construction throughput	15.516 candidates/s
End-to-end wall time	3,046.620 s (50.78 min)
End-to-end throughput	7.834 candidates/s
Median repository time	9.562 s
P90 repository time	203.008 s
P95 repository time	236.191 s

5.3 Scalability

The full-corpus timing experiment was executed using one analysis process with no worker pool. Candidate extraction and graph construction were timed without tracemalloc so that profiler overhead did not inflate the reported wall-clock measurements.

Table 9 summarizes the completed run.

The per-repository timing distribution is strongly skewed. Although the median combined analysis time is 9.562 seconds, P90 and P95 increase to 203.008 and 236.191 seconds, respectively. Reporting these quantiles is important because an arithmetic mean alone would obscure the large-repository tail.

The completed timing run predates formal process-level peak-RSS collection. We therefore do not report a peak-memory result for this run. A separate earlier tracemalloc experiment on the frozen 531-candidate pilot cohort measured Python allocation behavior, but its profiler-inflated

295 wall time is not comparable with the ordinary full-corpus timing experiment and is not used in the
 296 main scalability result.

297 5.4 Population and Reproducibility Notes

299 The full-corpus measurements above must not be compared directly with the historical 531-
 300 candidate frozen pilot cohort as though repository count were the only changing variable. The
 301 pilot manifest was generated by an earlier scanner configuration. Applying the current scanner
 302 to the same five frozen repository snapshots yields 508 candidates because subsequent language
 303 scoping and AST filtering changed candidate extraction.

304 We therefore preserve the three populations separately:

- 305 • **Frozen pilot cohort:** 531 candidates from five repositories, retained for compatibility with
 306 historical pilot and annotation studies;
- 307 • **Current five-repository scan:** 508 candidates on the same repository snapshots, used
 308 only to document scanner-version drift; and
- 309 • **Current full-corpus run:** 23,866 candidates from all 67 frozen repositories, used for the
 310 corpus-wide characterization and scalability results reported above.

311 None of these candidate populations constitutes a vulnerability count, and this descriptive
 312 corpus-wide experiment does not establish repository-wide recall.

314 6 Inventory-Bounded Evaluation

316 The main evaluation uses corpus-wide descriptive measurements, reproduction-confirmed behavior
 317 evidence, and a held-out representation-preservation study. We additionally retain an auxiliary
 318 source-level operation inventory to provide bounded evidence about operation recovery on code
 319 that was separated from development-time method tuning.

320 This evaluation is deliberately described as *inventory-bounded*. The finite inventory does not
 321 establish that every relevant operation in either repository has been enumerated and therefore
 322 cannot support claims of repository-wide exhaustive recall.

324 6.1 Held-Out Operation Inventory

325 The frozen inventory contains 344 source-level operation records from two repositories:

- 326 • `dddabtc/winremote-mcp`: 148 operation records; and
- 327 • `nexu-io/html-anything`: 196 operation records.

328 The two repositories were held separate from development-time method tuning when this
 329 inventory was constructed. The inventory covers eight security-sensitive operation categories.

330 The category distribution is not intended to estimate the prevalence of security-sensitive opera-
 331 tions in the broader LLM-agent ecosystem. It describes only the frozen auxiliary inventory used for
 332 this bounded evaluation.

334 6.2 Reference Evidence

335 Inventory entries are tied to frozen source locations and are reviewed using inspectable source-level
 336 evidence. Where stronger validation is available for a corresponding behavior, the evaluation
 337 distinguishes among the following evidence levels:

- 339 • **Source-level evidence**, establishing that a relevant program construction or dependency
 340 exists in the frozen source snapshot;
- 341 • **Stub or monkeypatch evidence**, establishing which call or argument is produced under a
 342 controlled input;

Table 10. Category distribution of the frozen 344-operation held-out inventory. The inventory is finite and is not treated as exhaustive repository-wide ground truth.

Category	Count
Command execution	76
Filesystem write	62
External tool invocation	58
Network access	46
Filesystem read	43
Browser control	27
Dynamic code execution	17
Filesystem delete	15
Total	344

- **Isolated execution evidence**, establishing that the corresponding external effect occurs in a controlled environment; and
- **Attack–defense paired evidence**, evaluating whether a candidate protection mechanism constrains the corresponding behavior.

These evidence levels are not interchangeable. In particular, the 344-operation inventory primarily serves as a source-level operation inventory and is not itself a set of confirmed vulnerabilities. Model-generated judgments are not treated as reference labels.

6.3 Prediction-to-Reference Matching

Method outputs are matched to inventory entries using the frozen matching rule

$$\text{match} = \text{repository} + \text{commit} + \text{file} + \text{operation category} + \text{line overlap}. \quad (2)$$

No label-dependent line tolerance or manual per-item matching is introduced after evaluation. A reference inventory record is considered recovered only when a method output satisfies the frozen matching rule.

Let I denote the 344-operation reference inventory and let M denote the set of method outputs matched to records in I . We define inventory-bounded recall as

$$R_{\text{inv}} = \frac{|M|}{|I|}. \quad (3)$$

Because I is a finite reviewed inventory rather than an exhaustive enumeration of all possible security-relevant operations in the two repositories, R_{inv} must not be interpreted as repository-wide recall.

6.4 Evaluation Status and Claim Boundary

The frozen inventory defines the finite population and the coordinate-level matching protocol used by the evaluator. At the time of submission, however, the independently adjudicated label file does not cover all 344 inventory items. The evaluator therefore refuses to emit aggregate or category-level recovery metrics, and we do not report matched counts, recall, precision, or F1 for this population. Results from the smaller reviewed sample are not substituted for full-inventory measurements. The inventory is retained here to document the evaluation population and protocol; completing the independent adjudication remains future work.

6.5 Interpretation and Limitations

This auxiliary evaluation serves a narrower purpose than the corpus-wide characterization in Section 5. The latter measures how the current analyzer behaves over all 67 frozen repositories, whereas the 344-operation inventory provides a finite source-level denominator on two held-out repositories.

The inventory therefore supports statements about recovery relative to its reviewed records, but not claims of exhaustive repository-wide recall. Likewise, an operation matched in this evaluation is not necessarily agent-controlled, exploitable, or vulnerable. Behavior-level and vulnerability conclusions remain governed by the separate reproduction-backed evidence layer described in the main paper.